# Vehicle Parking Management System (VPMS)

CSC2104: Object Oriented Programming in Java • Complete Viva Defense, OOP Code Mapping & Project Pitch Guide

Java 21 LTS | Spring Boot 3.3.4 | MySQL 8.0 (InnoDB) | React 19 + TypeScript | Full 4 OOP Pillars | Multithreading & Streams

## PART 1: HOW TO PITCH THE PROJECT (WORD-FOR-WORD SCRIPTS)

### 1.1 The 30-Second Elevator Pitch

`Opening Response`

**When the examiner says:** *"Introduce your project in 30 seconds."*

**SAY THIS:**

"Good morning/afternoon, Sir. Our project is a **Smart Vehicle Parking Management System** built with **Java 21, Spring Boot, MySQL, and a modern React frontend**.
Rather than treating parking as simple database records, we designed the entire system around **Object-Oriented Programming (OOP) principles**. We modeled real-world entities—like Vehicles, Parking Slots, and Tickets—into a strict object hierarchy. This enforces business rules such as automatic vehicle-type slot allocation, thread-safe ticket generation, and dynamic polymorphic tariff calculation with zero code duplication."

### 1.2 The 3-Minute Formal Presentation Pitch

`Full Viva Walkthrough`

**Step 1: The Domain Problem (30 sec)**

**SAY THIS:**

"In real-world commercial parking lots, manual operations suffer from slot hoarding, slow cashier lines, and pricing mistakes. Different vehicle classes—such as Two-Wheelers, Sedans, and Commercial Trucks—have distinct spatial footprints and different hourly tariffs."

**Step 2: The Core OOP Engineering Solution (90 sec)**

**SAY THIS:**

"To solve this cleanly, we engineered our Java backend around the **4 Core Pillars of OOP**:
• **Abstraction**: We created pure interface contracts ( `ParkingFeeCalculator` and `PrintableReceipt` ) and an abstract base class `Vehicle` to hide complexity.
• **Inheritance**: We established a clean hierarchy from `Vehicle` into `TwoWheeler` and `FourWheeler` , which branch into concrete classes `Bike` , `Car` , and `Truck` , chaining constructors via `super()` .
• **Polymorphism**: When a vehicle checks out, the service calls `vehicle.calculateParkingFee(duration)` . The JVM dynamically dispatches this call to the concrete vehicle class at runtime, completely eliminating ugly if-else or switch ladders.
• **Encapsulation**: All entity states are private and access-controlled with validation mutators in `ParkingSlot` and `ParkingSession` ."

**Step 3: Enterprise Features & Multithreading (45 sec)**

**SAY THIS:**

"Beyond basic OOP, we implemented background **Java Multithreading** via `ParkingMonitor` for continuous slot telemetry, a thread-safe synchronized producer-consumer queue for vehicle requests, and **Java File Streams & Serialization** for automated audit exports and system state recovery."

### 1.3 Pricing Model & Business Rules

`model/Vehicle.java & model/ParkingSession.java`

Our project uses **Tiered Hourly Pricing with a 1-Hour Minimum** (standard commercial lot model):

| Vehicle Subclass | Category Base | First Hour Rate | Each Additional Hour | OOP Implementation |
|---|---|---|---|---|
| **Bike** | TwoWheeler | ₹20.00 | ₹10.00 / hr | Overrides `calculateParkingFee()` in `Bike.java` |
| **Car** | FourWheeler | ₹30.00 | ₹20.00 / hr | Overrides `calculateParkingFee()` in `Car.java` |
| **Truck** | FourWheeler | ₹50.00 | ₹30.00 / hr | Overrides `calculateParkingFee()` in `Truck.java` |

**Duration Calculation:** Parked time is calculated via `Duration.between(entryTime, exitTime)` . Fractional hours round up using `Math.ceil(minutes / 60.0)` with a minimum of 1 hour.

## PART 2: DEEP-DIVE INTO ALL 4 OOP CONCEPTS IN THIS PROJECT

### 1. Encapsulation (Data Hiding & Controlled Access)

`model/ParkingSlot.java • model/ParkingSession.java`

**Definition:** Bundling state (variables) and behavior (methods) together while restricting direct access to object internals using `private` access modifiers and public getters/setters with validation.

**Where it is in our code:** In `ParkingSlot.java` and `ParkingSession.java`, all fields are strictly `private`. Slot status transitions (e.g. from `AVAILABLE` to `OCCUPIED`) cannot be set arbitrarily; they are modified through business methods.

```
// From model/ParkingSlot.java:
@Entity
@Table(name = "parking_slots")
public class ParkingSlot {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long slotId;                    // Private: inaccessible from outside

    @Column(nullable = false, unique = true)
    private String slotNumber;              // e.g. "C-01", "B-03"

    @Enumerated(EnumType.STRING)
    private SlotStatus status = SlotStatus.AVAILABLE;

    // Controlled getter & setter enforcing encapsulation
    public SlotStatus getStatus() { return this.status; }

    public void setStatus(SlotStatus status) {
        if (status == null) {
            throw new IllegalArgumentException("Slot status cannot be null");
        }
        this.status = status;
    }
}
```

**VIVA DEFENSE LINE:**
"We enforced Encapsulation by making all entity variables private. If outside classes could directly manipulate `slot.status = OCCUPIED` without booking a session, our database and telemetry would corrupt. Encapsulation guarantees valid state transitions."

### 2. Abstraction (Hiding Complexity via Contracts)

`interfaces/ParkingFeeCalculator.java • model/Vehicle.java`

**Definition:** Representing essential features without including background implementation details. Achieved in Java via **Interfaces** (100% abstract contract) and **Abstract Classes** (shared state + abstract method contracts).

**Where it is in our code:**
• `ParkingFeeCalculator.java`: Pure interface contract defining `calculateParkingFee()`.
• `Vehicle.java`: Abstract base class marked `public abstract class Vehicle implements ParkingFeeCalculator, PrintableReceipt`. It cannot be instantiated with `new Vehicle()`.

```
// From interfaces/ParkingFeeCalculator.java:
public interface ParkingFeeCalculator {
    double calculateParkingFee(long durationHours);
    double calculateParkingFee(long durationHours, double discountPercentage);
}

// From model/Vehicle.java:
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "vehicle_type")
public abstract class Vehicle implements ParkingFeeCalculator, PrintableReceipt {
    private String vehicleNumber;
    private String ownerName;

    // Abstract method: subclasses MUST implement their own logic!
    public abstract String getVehicleType();
}
```

**VIVA DEFENSE LINE:**
"Abstraction allows our checkout service to interact purely with the high-level `ParkingFeeCalculator` interface contract without knowing or caring how a Bike, Car, or Truck computes its fees under the hood."

### 3. Inheritance (Code Reuse & Class Hierarchies)

`model/TwoWheeler.java` • `model/Car.java` • `model/Truck.java`

**Definition:** The mechanism by which a child class acquires the properties and methods of a parent class using the `extends` keyword, enabling code reusability and hierarchical categorization.

**Our Project Implements Two Forms of Inheritance:**

- **Multilevel Inheritance:** `Vehicle` (Base) → `TwoWheeler` (Intermediate) → `Bike` (Leaf)
- **Hierarchical Inheritance:** `Vehicle` → `FourWheeler` → `Car` and `Truck`
- **Constructor Chaining with `super()`:** Child constructors invoke parent constructors using `super(vehicleNumber, ownerName, phone)`.

```
// Multilevel Hierarchy Example:
public abstract class TwoWheeler extends Vehicle {
    public TwoWheeler(String number, String owner, String phone) {
        super(number, owner, phone); // Chaining to Vehicle constructor
    }
}

public class Bike extends TwoWheeler {
    public Bike(String number, String owner, String phone) {
        super(number, owner, phone); // Chaining to TwoWheeler constructor
    }
}

// Hierarchical Hierarchy Example:
public class Car extends FourWheeler { ... }
public class Truck extends FourWheeler { ... }
```

**VIVA DEFENSE LINE:**

"We used both Multilevel and Hierarchical inheritance. Properties like `vehicleNumber` and `ownerName` are defined once in `Vehicle` and inherited by all subclasses. Subclasses call `super()` to chain constructor initialization."

### 4. Polymorphism (One Name, Many Forms)

`service/ParkingService.java` • `model/Car.java` • `model/Bike.java`

**Definition:** The ability of a message, method, or object to be processed in more than one form.

**Our Project Implements Both Types of Polymorphism:**

**A. Compile-Time Polymorphism (Method Overloading):** Same method name in the same class/interface with different parameter signatures.

```
// Overloading in Vehicle.java:
public double calculateParkingFee(long durationHours) { ... }
public double calculateParkingFee(long durationHours, double discountPercentage) { ... }
public double calculateParkingFee(Integer hoursWrapper) { ... } // Wrapper autoboxing
```

**B. Runtime Polymorphism (Method Overriding & Dynamic Method Dispatch):** A subclass provides a specific implementation of a method already declared in its superclass using `@Override`.

```
// In service/ParkingService.java:
// The reference variable type is abstract `Vehicle`:
Vehicle vehicle = session.getVehicle();

// At runtime, the JVM uses Dynamic Method Dispatch to invoke the OVERRIDDEN
// method in Bike, Car, or Truck based on the actual object created in memory!
double fee = vehicle.calculateParkingFee(durationHours);
```

**VIVA DEFENSE LINE:**

"Runtime Polymorphism is demonstrated at checkout. `ParkingService` only holds a reference of type `Vehicle`. When `vehicle.calculateParkingFee()` is called, the JVM resolves the exact child implementation (Car at ₹30/hr vs Bike at ₹20/hr) via dynamic method dispatch."

## PART 3: ADDITIONAL ADVANCED OOP CONCEPTS IN OUR REPOSITORY

### 3.1 Multiple Interface Implementation & Default Methods

`interfaces/PrintableReceipt.java`

Java does not support multiple inheritance with classes (to prevent the Diamond Problem), but supports it through **Interfaces**. `Vehicle` implements both `ParkingFeeCalculator` and `PrintableReceipt`.

`PrintableReceipt` also features a **Java 8 `default` method** providing a ready-to-use receipt footer that implementing classes inherit automatically without rewriting.

### 3.2 final Keyword, static Members & Constants

`model/Car.java` • `model/Bike.java` • `model/Truck.java`

• `public static final double CAR_FIRST_HOUR_RATE = 30.0;` : Demonstrates constant declaration. `final` prevents reassignment; `static` allocates memory once per class rather than per object instance.

### 3.3 Multithreading & Thread Synchronization

`threads/ParkingMonitor.java` • `threads/ParkingRequestQueue.java`

• **Approach 1 (Extending Thread):** `ParkingMonitor extends Thread` with custom `run()` loop, daemon flag, and thread priority setting.

• **Approach 2 (Implementing Runnable):** `ParkingSessionMonitor implements Runnable` for decoupled background tasks.

• **Thread Synchronization:** `ParkingRequestQueue.java` implements the classic **Producer-Consumer Pattern** using `synchronized` methods, `wait()`, and `notifyAll()` to safely enqueue vehicle booking requests.

### 3.4 Custom Exception Handling

`exception/SlotFullException.java` • `exception/VehicleNotFoundException.java`

Custom user-defined exceptions extending `RuntimeException` (Unchecked) with global REST error interception via `@RestControllerAdvice` returning structured HTTP JSON payloads.

### 3.5 File I/O & Object Serialization

`io/ParkingBackupManager.java` • `io/ParkingReportExporter.java`

• **Byte Streams:** `FileOutputStream` and `FileInputStream` for binary backups ( `parking_backup.dat` ).

• **Character Streams:** `BufferedWriter` and `FileWriter` for readable text audit reports ( `parking_history.txt` ).

## PART 4: TOP 20 EXAMINER VIVA QUESTIONS & BULLETPROOF MODEL ANSWERS

**Q1: What is the main objective of your project, and what is the tech stack?**

**Answer:** Our project is an automated Vehicle Parking Management System that manages slot allocation, vehicle check-ins, automated billing, and live telemetry. The tech stack is **Java 21 LTS, Spring Boot 3.3.4, MySQL 8.0 with Hibernate ORM, and React 19 with TypeScript and Tailwind CSS**.

ARCHITECTURE

**Q2: Where is Abstraction used in your codebase?**

**Answer:** In two places: (1) Interface contracts like `ParkingFeeCalculator` and `PrintableReceipt`, and (2) Abstract classes like `Vehicle`, `TwoWheeler`, and `FourWheeler`. The abstract class defines common state (plate number, entry time) and forces subclasses to implement abstract methods like `getVehicleType()`.

ABSTRACTION

**Q3: Why did you make Vehicle an abstract class instead of a regular class or an interface?**

**Answer:** An interface cannot hold concrete instance state (like vehicle registration number or entry timestamps) that all vehicles share. A regular concrete class would allow someone to create a generic `new Vehicle()`, which makes no sense in the real world—every vehicle in a parking lot is a specific type like a Car or Bike. An abstract class is the exact OOP design pattern for shared state with enforced subclass specialization.

DESIGN PATTERNS

**Q4: Show me an example of Runtime Polymorphism in your code.**

**Answer:** In `ParkingService.java` at line 171 during checkout: `double fee = vehicle.calculateParkingFee(duration);`. The reference type is the abstract base class `Vehicle`, but at runtime the JVM uses Dynamic Method Dispatch to call the overridden `calculateParkingFee()` in `Car`, `Bike`, or `Truck` based on the actual object in memory.

POLYMORPHISM

**Q5: What is the difference between Method Overloading and Method Overriding? Where did you use them?**

**Answer: Overloading (Compile-Time)** occurs in the same class/interface where methods share the same name but have different parameter lists (e.g. `calculateParkingFee(long)` vs `calculateParkingFee(long, double)` in `ParkingFeeCalculator`). **Overriding (Runtime)** occurs when a subclass redefines a method from its superclass with the exact same signature using `@Override` (e.g. `Car.java` overriding `Vehicle.calculateParkingFee`).

POLYMORPHISM

**Q6: How did you implement Encapsulation?**

**Answer:** All entity fields in `ParkingSlot.java`, `ParkingSession.java`, and `Vehicle.java` are declared `private`. State changes are guarded through getters and validation setters. For example, setting slot status cannot be done directly; it requires passing non-null status checks and checking out active sessions.

ENCAPSULATION

**Q7: What types of Inheritance are implemented in this project?**

**Answer:** We implemented two forms: (1) **Multilevel Inheritance:** `Vehicle` → `TwoWheeler` → `Bike`. (2) **Hierarchical Inheritance:** `Vehicle` → `FourWheeler` → `Car` and `Truck`. Both chain constructors using the `super` keyword.

INHERITANCE

**Q8: Why doesn't Java support Multiple Inheritance of classes? How did you overcome this?**

**Answer:** Java disallows multiple class inheritance to avoid the **Diamond Problem** (ambiguity when two parent classes define the same method). We achieved multiple inheritance of type through **Interfaces**: `Vehicle` implements both `ParkingFeeCalculator` and `PrintableReceipt`.

INTERFACES

**Q9: What is the purpose of the super keyword?**

**Answer:** The `super` keyword is used in subclasses to refer directly to the immediate superclass. In our project, it is used for **Constructor Chaining**: `Car(...)` calls `super(vehicleNumber, ownerName, phone)` to pass initialization parameters up to `FourWheeler` and `Vehicle`.

INHERITANCE

**Q10: Where is Multithreading used in your project?**

**Answer:** We implemented multithreading in two ways: (1) `ParkingMonitor.java` extends `java.lang.Thread` to run background slot occupancy telemetry at set intervals. (2) `ParkingSessionMonitor.java` implements `Runnable`. (3) `ParkingRequestQueue.java` uses synchronized blocks with `wait()` and `notifyAll()` to implement a producer-consumer request buffer.

MULTITHREADING

**Q11: What is the difference between Thread.start() and Thread.run()?**

**Answer:** Calling `run()` simply executes the method in the current calling thread synchronously without creating a new thread. Calling `start()` creates a new OS thread of execution and schedules it in the JVM thread scheduler, which then calls `run()` asynchronously.

MULTITHREADING

**Q12: How do you handle concurrency if two users try to book the last remaining slot simultaneously?**

**Answer:** In Spring Boot, the allocation method in `ParkingService.java` uses `@Transactional` with database row-level locking. Furthermore, our `ParkingLot` manager uses Java synchronized locks on the slot allocation matrix, preventing race conditions. If no slot is free, a custom `SlotFullException` is thrown.

CONCURRENCY & TRANSACTIONS

**Q13: What custom exceptions did you create?**

**Answer:** We created `SlotFullException` (thrown when attempting to park in an occupied/full lot) and `VehicleNotFoundException` (thrown when checking out an unregistered or already departed vehicle). Both inherit from `RuntimeException` and are caught globally by `GlobalExceptionHandler`.

EXCEPTION HANDLING

**Q14: How does File I/O work in your project?**

**Answer:** We implemented two I/O mechanisms in `io/`: (1) `ParkingReportExporter` uses character-based streams (`BufferedWriter`, `FileWriter`) to export human-readable text audit reports. (2) `ParkingBackupManager` uses byte-based streams (`FileOutputStream`, `FileInputStream`) for raw binary snapshots.

FILE HANDLING

**Q15: What is the difference between checked and unchecked exceptions?**

**Answer:** Checked exceptions (inheriting directly from `Exception`, like `IOException` in our File I/O classes) must be declared with `throws` or handled in a `try-catch` block at compile time. Unchecked exceptions (inheriting from `RuntimeException`, like our `SlotFullException`) do not require explicit compile-time handling.

EXCEPTIONS

**Q16: Where are your SQL queries written in Spring Boot?**

**Answer:** Spring Data JPA abstracts raw SQL away. In `ParkingSlotRepository` and `ParkingSessionRepository`, we extend `JpaRepository<T, ID>`. Spring generates the underlying SQL queries automatically at runtime through derived query methods (like `findByStatusAndSlotType()`) and Hibernate ORM.

SPRING DATA & ORM

**Q17: Is there any database limit since you are using MySQL?**

**Answer:** No artificial limit. We use MySQL 8.0 with the InnoDB storage engine, which supports table sizes up to **64 Terabytes** and billions of rows. Primary keys are 64-bit `BIGINT` (`Long` in Java), which can store up to 9 quintillion records. Concurrency is handled by Spring Boot's built-in **HikariCP** connection pool.

DATABASE ARCHITECTURE

**Q18: What is the purpose of the volatile keyword in ParkingMonitor?**

**Answer:** In multithreading, CPU cores cache variables locally in their hardware caches. Declaring `private volatile boolean running = true;` guarantees **memory visibility** across all threads—any write to `running` is flushed directly to main RAM so the monitoring thread sees the stop flag immediately.

MEMORY & THREADS

**Q19: What is the difference between final, finally, and finalize in Java?**

**Answer:** `final` is a keyword to declare constants or prevent method overriding / class inheritance. `finally` is a block in exception handling that executes regardless of whether an exception was thrown. `finalize()` was an Object method invoked by the Garbage Collector before object reclamation (deprecated in modern Java).

JAVA KEYWORDS

**Q20: Why did you decouple the Frontend (React) and Backend (Spring Boot)?**

**Answer:** This follows the **Single Responsibility Principle** and modern Client-Server Architecture. The Spring Boot backend acts as a stateless RESTful API exposing JSON endpoints, while React handles client-side state, responsiveness, and spatial rendering. This allows the backend to serve mobile apps or IoT gate barriers in the future without changing business logic.

SYSTEM DESIGN

## PART 5: IDE "SHOW ME THE CODE" NAVIGATION MAP

| OOP Concept | File Path on Disk | Key Classes / Methods to Point to |
|---|---|---|
| **Abstraction** | `backend/src/main/java/com/parking/management/model/Vehicle.java` | `public abstract class Vehicle implements ParkingFeeCalculator` |
| **Interfaces** | `backend/src/main/java/com/parking/management/interfaces/ParkingFeeCalculator.java` | `double calculateParkingFee(long durationHours);` |
| **Inheritance** | `backend/src/main/java/com/parking/management/model/Car.java` & `Bike.java` | `public class Car extends FourWheeler`, `super(...)` constructor chaining |
| **Polymorphism** | `backend/src/main/java/com/parking/management/service/ParkingService.java` (L171) | `vehicle.calculateParkingFee(durationHours);` (Dynamic dispatch) |
| **Encapsulation** | `backend/src/main/java/com/parking/management/model/ParkingSlot.java` | Private fields with controlled mutators ( `getStatus()`, `setStatus()` ) |
| **Multithreading** | `backend/src/main/java/com/parking/management/threads/ParkingMonitor.java` | `extends Thread`, `run()`, `setPriority()`, `setDaemon(true)` |
| **Synchronization** | `backend/src/main/java/com/parking/management/threads/ParkingRequestQueue.java` | `synchronized` enqueue/dequeue with `wait()` and `notifyAll()` |
| **Exceptions** | `backend/src/main/java/com/parking/management/exception/SlotFullException.java` | Custom unchecked exception with HTTP 409 Conflict status |
| **File I/O** | `backend/src/main/java/com/parking/management/io/ParkingBackupManager.java` | `FileOutputStream` and `FileInputStream` byte streams |